

05 — Dati che vengono da fuori: `fetch`, cache e migrazioni IndexedDB

Come si aggiunge una chiamata di rete a un'app nata per funzionare senza rete, senza tradirne la promessa. Il caso concreto è la quotazione delle carte. Esempi: `src/data/prezzi.js`, `src/data/deposito.js`, `src/data/rarita.js`.

1. La regola che si sta per rompere

Fino a qui l'app non chiamava mai la rete di sua iniziativa: i dati delle carte sono JSON nel repository, il service worker li mette in cache, e offline funziona tutto. È una promessa forte, e i prezzi la mettono in crisi — un prezzo committato nel repo sarebbe già vecchio il giorno dopo.

La soluzione non è “chiamare la rete quando serve”, che è il modo in cui le app diventano inutilizzabili in cantina. È **spostare la decisione all'utente**:

Vincolo	Come si mantiene
Niente rete a sorpresa Offline resta usabile	Si scarica solo al tocco di “Calcola quotazione” Ogni prezzo va in IndexedDB con la sua data e si rimostra sempre
Il fallimento non è un errore	Senza rete si tiene l'ultimo prezzo noto e lo si dichiara
Costo controllato	Tetto di 60 carte per volta: una richiesta per carta

Il risultato è che la funzione nuova non toglie niente a chi è offline: vede i prezzi vecchi, etichettati come vecchi.

2. `fetch` non lancia eccezioni per un 404

L'errore classico di chi arriva da un client HTTP Java: `fetch()` **rifiuta la Promise solo se la richiesta non parte o non arriva** (rete assente, DNS, CORS). Un 404 o un 500 sono risposte perfettamente riuscite dal punto di vista di `fetch`, e vanno controllate a mano:

```
const risposta = await fetch(`${API}/${idSet}-${numeroApi}`);  
if (!risposta.ok) throw new Error(`HTTP ${risposta.status}`);
```

Senza quella riga, `risposta.json()` proverebbe a leggere la pagina d'errore e il difetto salterebbe fuori molto più lontano dalla sua causa.

Parallelismo con un tetto

Sessanta richieste sparate insieme sono un modo per farsi limitare dall'API. Il modulo usa lo stesso schema di `tools/scarica-set.mjs`: N “operai” che pescano da una coda condivisa finché non è vuota.

```
const operai = Array.from({ length: PARALLELE }, async () => {  
  while (daFare.length) { /* ... */ }  
});  
await Promise.all(operai);
```

Differenza con Java: sembra un thread pool, non lo è. C'è una sola thread: ogni `await` restituisce il controllo al ciclo di eventi, e gli “operai” si alternano nei buchi di attesa della rete. Proprio perché la thread è una sola, `daFare.shift()` non ha bisogno di nessun `synchronized` — fra due `await` il codice non può essere interrotto a metà.

3. Migrare lo schema di IndexedDB

I prezzi vogliono un object store nuovo. In IndexedDB la struttura si tocca **solo** dentro `onupgradeneeded`, che scatta quando il numero di versione del codice supera quello sul disco:

```
const VERSIONE_DB = 3; // era 2
// ...
if (daVersione < 1) { /* collezione */ }
if (daVersione < 2) { /* mazzi */ }
if (daVersione < 3) { db.createObjectStore(STORE_PREZZI, { keyPath: 'id' }); }
```

I tre `if` **a cascata, senza** `else`, sono la parte che conta: chi installa oggi da zero parte da `oldVersion = 0` ed esegue tutti e tre i passi; chi ha l'app da mesi entra con `oldVersion = 2` ed esegue solo l'ultimo, **tenendo la sua collezione**. Un `else if` avrebbe creato lo store dei prezzi solo ai nuovi.

Vale anche la regola opposta: uno store separato invece di un campo `prezzo` dentro la riga della carta. I due dati hanno cicli di vita diversi — le quantità possedute sono la verità dell'utente, i prezzi sono una copia di comodo — e “cancella tutti i prezzi” non deve poter sfiorare la collezione.

4. Normalizzare i dati altrui prima di mostrarli

TCGdex dà la rarità come testo libero, e nei dati italiani convivono "Comune", "Olografica Rara V", "deux Étoiles" e "Une Diamant": 35 valori distinti, con il francese non tradotto. Messi in un menu così, sono inutili.

`src/data/rarita.js` li riduce a una dozzina di **classi ordinate**, con una regola (non un elenco chiuso) per riconoscerle:

```
{ codice: 'stella-2', etichetta: ' Due stelle',
  prova: (r) => /^deux [ée]toiles?$|^deux chromatique$/i.test(r) }
```

La differenza fra regola ed elenco si vede al prossimo set: i nomi nuovi arrivano di continuo («Rara illustrazione speciale» non esisteva tre anni fa). Chi non rientra in nessuna classe finisce in *altra*, e c'è un **test sui dati veri** che fallisce se quel gruppo non è vuoto: è il modo di accorgersene adesso invece che da una collezione che ha smesso di trovarsi.

5. Esercizi

1. `aggiornaPrezzi()` prende le voci di collezione intere e non solo `numero`, perché la riga salvata può dire 11 mentre il dataset e l'API dicono 011. Cosa succedeva prima della correzione? Perché il difetto si vedeva su cinque carte e non su tutte?
2. Il tetto di 60 carte per volta è in `MASSIMO_PER_VOLTA`. Immagina di volerlo togliere: quali due cose andrebbero cambiate perché quotare 3.000 carte non diventi un disastro (per l'API e per l'utente che guarda lo schermo)?
3. Nel service worker, le richieste ad `api.tcgdex.net` non sono intercettate, mentre quelle ad `assets.tcgdex.net` sì. Perché le immagini si possono mettere in cache HTTP e i prezzi no?